

Lab 5: Mastering Chat Variables, Slash Commands, and Chat Participants

Objective

Master the use of GitHub Copilot's chat features including chat variables (#), slash commands (/), and chat participants (@) to enhance your coding workflow. Learn how to provide precise context, execute common tasks efficiently, and leverage specialized AI agents for different development scenarios.

What You Will Learn in This Lab

- Using chat variables (#) to reference specific code context
- Executing slash commands (/) for common development tasks
- Working with chat participants (@) for specialized assistance
- Combining different chat features for complex workflows
- Understanding when and how to use each feature type
- Building efficient prompting strategies with structured commands

Prerequisites: VS Code with GitHub Copilot Chat enabled, and a codebase to work with (your own or the sample: <https://github.com/DevOnGlobal/ai-playground-exercises>)

Steps

Step 1: Project Setup

1. Open VS Code with a project loaded (use your own or clone the sample codebase)
2. Open the Chat view using `Ctrl + Alt + I` (Windows) or explore the Copilot icon in the Activity Bar
3. (Optional) Create a new file called `chat-practice-notes.md` to document your findings
4. Ensure you can see both the Chat panel and your code editor

Step 2: Exploring Chat Variables (#)

Chat variables let you reference specific parts of your code as context. Let's practice each one:

#file - Reference Current File

1. Open any code file in your project
2. In Copilot Chat, try these prompts:
 - `#file explain the main purpose of this file`

- `#file what are the potential security issues in this code?`
- `#file suggest improvements for better readability`

#selection - Reference Selected Code

1. Select a function or code block in your editor
2. Try these prompts:
 - `#selection explain what this code does step by step`
 - `#selection rewrite this to be more efficient`
 - `#selection what edge cases should I consider?`

#function - Reference Current Function

1. Place your cursor inside a function
2. Practice with:
 - `#function add comprehensive error handling`
 - `#function write unit tests for this function`
 - `#function optimize this for performance`

#class - Reference Current Class

1. Position cursor inside a class definition
2. Try:
 - `#class explain the design pattern used here`
 - `#class suggest additional methods that would be useful`
 - `#class review this class for SOLID principles`

#line - Reference Current Line

1. Position cursor on a specific line
2. Use:
 - `#line explain this syntax`
 - `#line is there a bug on this line?`
 - `#line suggest a better way to write this`

#project - Reference Project Context

1. Try these project-wide queries:

- `#project what's the overall architecture?`
- `#project find all API endpoints`
- `#project identify unused dependencies`

#block - Reference Current Code Block

1. Position cursor inside a code block (if/else, loop, etc.)

2. Practice:

- `#block simplify this logic`
- `#block what could go wrong here?`
- `#block add logging and monitoring`

Step 3: Mastering Slash Commands (/)

Slash commands provide shortcuts for common development tasks.

Basic Slash Commands

1. `/explain` - Understanding code
 - Try: `/explain` (with a file or selection active)
 - Practice: `/explain how does this algorithm work?`
2. `/fix` - Fixing problems
 - Intentionally introduce a syntax error in your code
 - Select the problematic code and use: `/fix`
 - Try: `/fix this function has a memory leak`
3. `/tests` - Generating tests
 - Select a function and use: `/tests`
 - Try: `/tests create comprehensive test cases`
 - Practice: `/tests include edge cases and error scenarios`
4. `/help` - Getting assistance
 - Try: `/help` to see available commands
 - Use: `/help explain chat variables`

Advanced Slash Commands

1. `/new` - Creating new projects
 - Try: `/new create a simple REST API`

- Practice: `/new build a React component for user authentication`
2. `/clear` - Starting fresh
 - Use: `/clear` to start a new chat session
 - Note how this clears conversation history
 3. `/fixTestFailure` - Debugging tests
 - If you have failing tests, try: `/fixTestFailure`
 - Practice: `/fixTestFailure why is my unit test not passing?`

Combining Slash Commands with Context

1. Try these combinations:
 - `/explain #selection in simple terms`
 - `/fix #function and add error handling`
 - `/tests #class with full coverage`

Step 4: Working with Chat Participants (@)

Chat participants are specialized AI agents for different domains.

@workspace - Codebase Expert

1. **Project Structure Questions:**
 - `@workspace how is this project organized?`
 - `@workspace what are the main components?`
 - `@workspace find all TODO comments`
2. **Code Relationships:**
 - `@workspace how do these classes interact?`
 - `@workspace where is this function called from?`
 - `@workspace show me the data flow through this application`
3. **Architectural Insights:**
 - `@workspace what design patterns are used?`
 - `@workspace identify potential refactoring opportunities`
 - `@workspace explain the dependency structure`

@vscode - VS Code Helper

1. **Editor Configuration:**

- @vscode how do I set up debugging for Node.js?
- @vscode configure auto-formatting for this project
- @vscode set up keyboard shortcuts for testing

2. Extensions and Features:

- @vscode recommend extensions for Python development
- @vscode how do I use the integrated terminal effectively?
- @vscode explain the source control features

3. Productivity Tips:

- @vscode show me time-saving shortcuts
- @vscode how to customize the editor for this language?

@terminal - Command Line Expert

1. Basic Commands:

- @terminal how do I create a git branch?
- @terminal find all files containing a specific string
- @terminal compress this directory

2. Development Workflows:

- @terminal set up a development environment
- @terminal deploy this application to production
- @terminal automate testing with scripts

3. System Administration:

- @terminal monitor system performance
- @terminal manage environment variables
- @terminal troubleshoot network connectivity

@github - GitHub Integration

1. Repository Management:

- @github what issues are assigned to me?
- @github show recent pull requests
- @github create a release checklist

2. Collaboration:

- @github review best practices for this repository

- @github suggest branch protection rules
- @github explain the contribution workflow

Step 5: Advanced Combinations and Workflows

Now let's combine different chat features for complex tasks.

Context-Rich Problem Solving

1. Debugging Workflow:

```
...@workspace identify the bug in #selection
.../fix the identified issue
.../tests create tests to prevent this bug
```

2. Feature Development:

```
...#project show me similar patterns for user authentication
.../new create a login component following these patterns
...@vscode set up debugging for this new feature
```

Code Review Process

1. Comprehensive Review:

```
#file review this for security issues
...@workspace check how this integrates with existing code
.../tests suggest additional test cases needed
```

2. Documentation Generation:

```
...#class generate API documentation
...@github create README section for this feature
.../explain how other developers should use this
```

Step 6: Building Efficient Prompting Strategies

Practice creating efficient, multi-part prompts.

Progressive Context Building

1. Start broad: @workspace what does this project do?

2. Get specific: `#file explain the main algorithm`
3. Take action: `/optimize #selection for better performance`

Task-Specific Workflows

1. New Feature Development:

- `@workspace show existing patterns for [feature type]`
- `/new create [feature] following these patterns`
- `/tests generate comprehensive test suite`
- `@vscode configure debugging for this feature`

2. Bug Investigation:

- `#selection explain what this code should do`
- `@workspace find similar code that works correctly`
- `/fix #selection based on working examples`
- `/tests create regression tests`

Step 7: Troubleshooting and Tips

Practice recovering from common issues and optimizing your workflow.

When Commands Don't Work

1. Try typing `/` in chat to see available commands
2. Use `#` to see available variables
3. Type `@` to see available participants
4. Use `/help` for command explanations

Optimizing Context

1. **Too Much Context:** Use specific variables like `#selection` instead of `#file`
2. **Too Little Context:** Combine multiple variables: `#function` and `#class`
3. **Wrong Context:** Use `@workspace` for project-wide questions

Step 8: Real-World Scenarios

Apply your skills to realistic development scenarios.

Scenario 1: Onboarding to New Codebase

1. `@workspace give me an overview of this project`

2. #project what are the key files I should understand first?
3. /explain #file (for each key file)
4. @vscode set up the development environment

Scenario 2: Adding New Feature

1. @workspace show me similar features in this codebase
2. /new create [feature] following existing patterns
3. /tests generate test cases for the new feature
4. @github create a pull request checklist

Scenario 3: Performance Optimization

1. #project identify performance bottlenecks
2. #selection optimize this slow function
3. /tests create performance benchmarks
4. @terminal set up performance monitoring

Step 9: Documentation and Knowledge Sharing

Use chat features to create documentation.

1. Generate Team Documentation:

- @workspace create onboarding guide for new developers
- #project document the architecture and design decisions
- /explain key algorithms for the team wiki

2. Create Development Guides:

- @vscode document the development setup process
- @terminal create deployment instructions
- @github document the code review process

Step 10: Assessment and Practice

Test your understanding with these challenges.

Challenge 1: Context Precision

For a single function, use three different variables (#function, #selection, #line) and compare the responses.

Challenge 2: Multi-Agent Workflow

Create a complete feature using at least three different participants (@workspace, @vscode, @terminal).

Challenge 3: Command Combinations

Solve a complex problem using at least five different slash commands in sequence.

Key Takeaways to Discuss

- **Chat variables** provide precise context control for better responses
- **Slash commands** streamline common development tasks
- **Chat participants** offer specialized expertise for different domains
- **Combining features** creates powerful, multi-step workflows
- **Context matters** - choose the right variable for the task
- **Progressive prompting** builds complex solutions step by step

Best Practices Learned

- Start with broad context (@workspace) then narrow down (#selection)
- Use slash commands for standard tasks, natural language for custom requests
- Combine multiple variables when you need broader context
- Match the participant to the domain (VS Code questions → @vscode)
- Use /clear to start fresh when conversations get complex

Next Steps

Practice these features in your daily development workflow. Try to replace manual tasks with appropriate chat commands, and experiment with different combinations to find your optimal productivity patterns.